PAGE No. 1
DATE : / 202

* Syntax in C / C++

- It is the set of rules and regulations that define how a program should be written in a programming language.

- Basically we can say that syntax is is the collection of grammatical rules that specify how programs must be written so that compiler can understand and execute them.

- Importance
  - allows the compiler to understand the program (i)
  - Improves code readability
  - Makes programs easy to maintain (ii)
  - Reduces programming mistakes (iii)

=> Basic structure in C

```
#include <stdio.h>

int main()
{
  print("My KCC");
  return 0;
}
```

output :- My KCC

(i) Header file

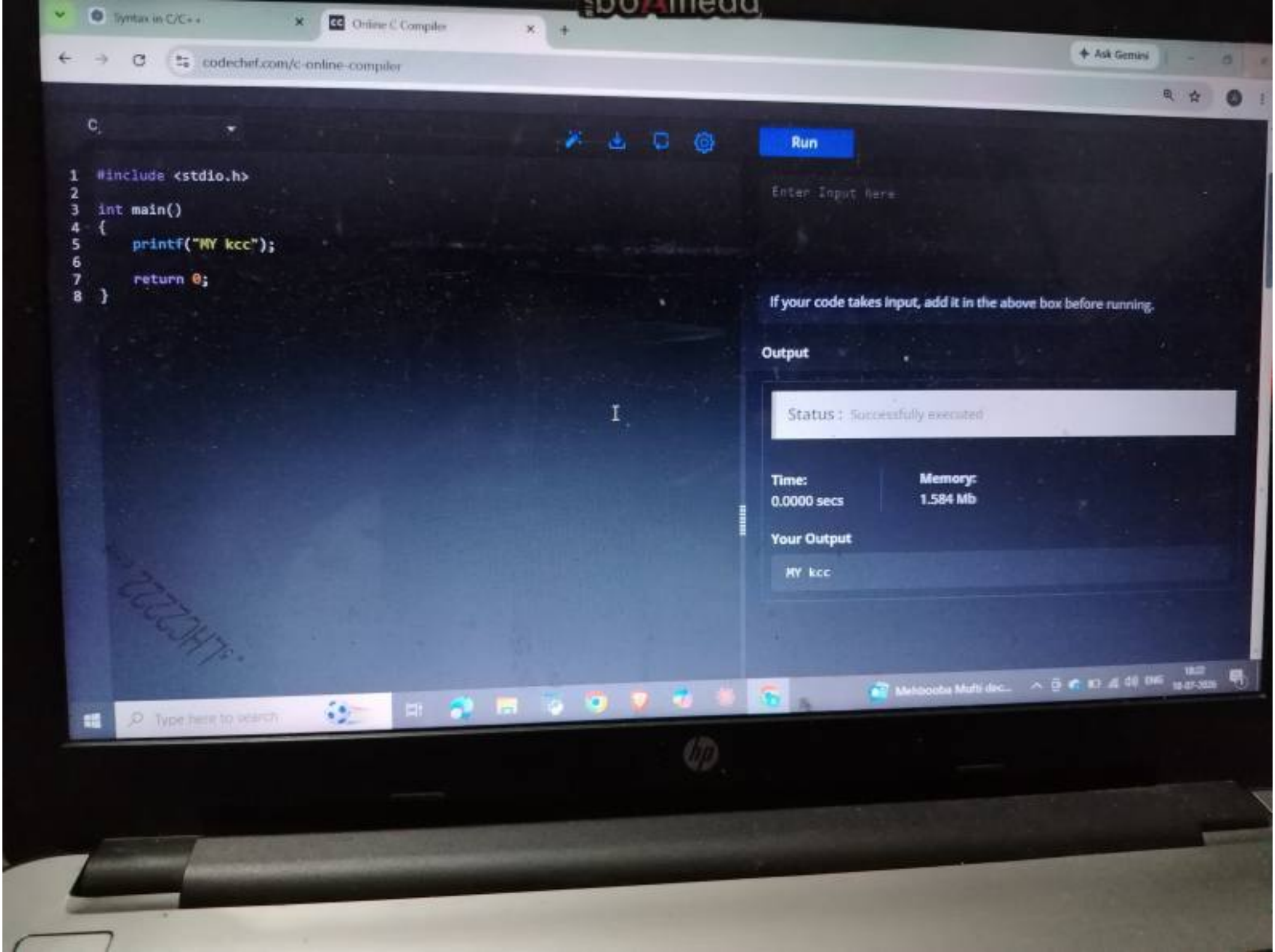
```
#include <stdio.h>
```

• Includes standard input library

(ii) Main function

```
int main()
```

every C program start executing from the main() function


Syntax in C/C++
Online C Compiler
codechef.com/c-online-compiler
Ask Gemini
C
Run
1 #include <stdio.h>
2
3 int main()
4 {
5 printf("MY kcc");
6
7 return 0;
8 }
Enter Input here
If your code takes input, add it in the above box before running.
Output
Status : Successfully executed
Time:
0.0000 secs
Memory:
1.584 Mb
Your Output
MY kcc
Type here to search

(iii) Curly braces

{ }

It define the beginning and ending of a block of code.

(iv) statements

printf("My KCC\n");

executable instruction = statement

(v) Return statement

return 0;

Tells that program has executed successfully.

-> Basic structure in C++

```
#include <iostream>
using namespace std;

int main()
{
  cout << "Hello world";

  return 0;

}
```

output :- Hello world

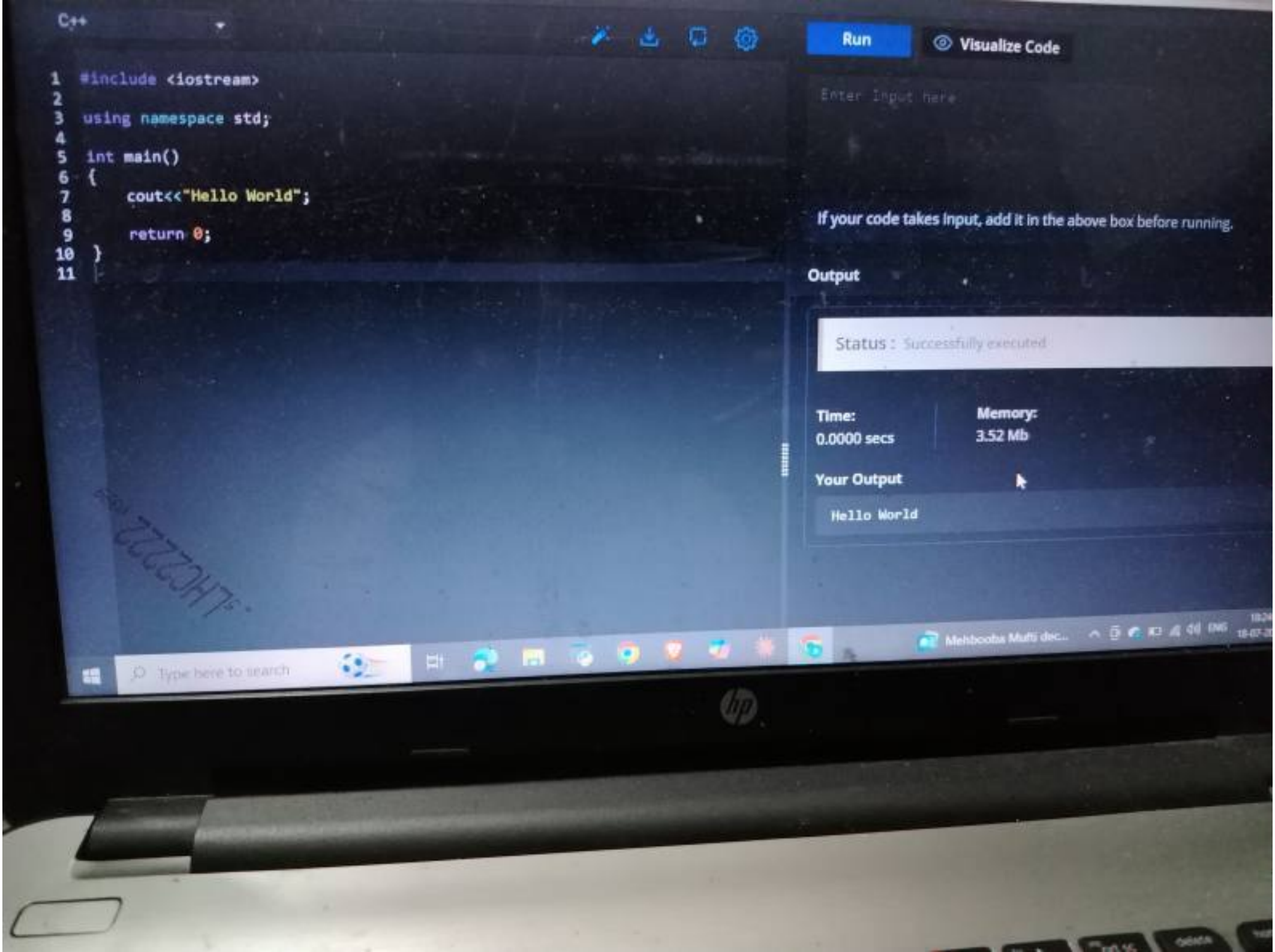
(i) Header file

#includ <iostream>

(ii) Namespace

using namespace std;

allow to write cout, cin, endl without std::


C++
Run
Visualize Code
1 #include <iostream>
2
3 using namespace std;
4
5 int main()
6 {
7 cout<<"Hello World";
8
9 return 0;
10 }
11
Enter Input here
If your code takes input, add it in the above box before running.
Output
Status : Successfully executed
Time:
0.0000 secs
Memory:
3.52 Mb
Your Output
Hello World
Type here to search

(iii) Main function

int main()

(iv) Output statement

cout << "KCC is best";

Displays output on screen

(v) Return statement

return 0;

Ends the program successfully

Rules of Syntax

(1) Every statement ends with a semicolon (;)

```
int a = 10;    // correct
int a = 10     // wrong
```

(2) Curly braces must match

```
{
  printf("My K");
}

  // correct

{
  printf("My K");

  # incorrect
```

(3) Parenthesis must match

Printf ("KC(JJ)"; // correct

printf ("KC("JJ"; // incorrect

(4) Keywords Must be written correctly

int number; // correct

INT number; // incorrect

PAGE No. 8
DATE :

Identifiers

- Identifiers are the names given to variables, functions, arrays & structures

eg
```
int marks;
float salary;
```

Here
- marks, salary are identifiers

- Rules for identifiers
  - Can contain letters
  - Can contain digits
  - Can contain underscore (_)
  - Cannot start with a digit
  - Can't contain spaces
  - Can't be a keyword

Correct

student
student1
student_name

Incorrect

1student
student-name
student name

-> Keywords

These are the words that are already defined in C / C++

eg
int , float , double , char ,
return , if , else , while ,

- These cannot be used as variable names

-> Data types

A datatypes tells about the type of data a variable can store

- integer

int age = 19 ;

stores whole numbers.

- Float

  float marks = 95.5;

  stores decimal numbers

- Double

  double pi = 3.14159265;

  Stores large deimal numbers

- Character

  char grade = 'A';

- Boolean

  bool result = true;

  Stores
  true
  false

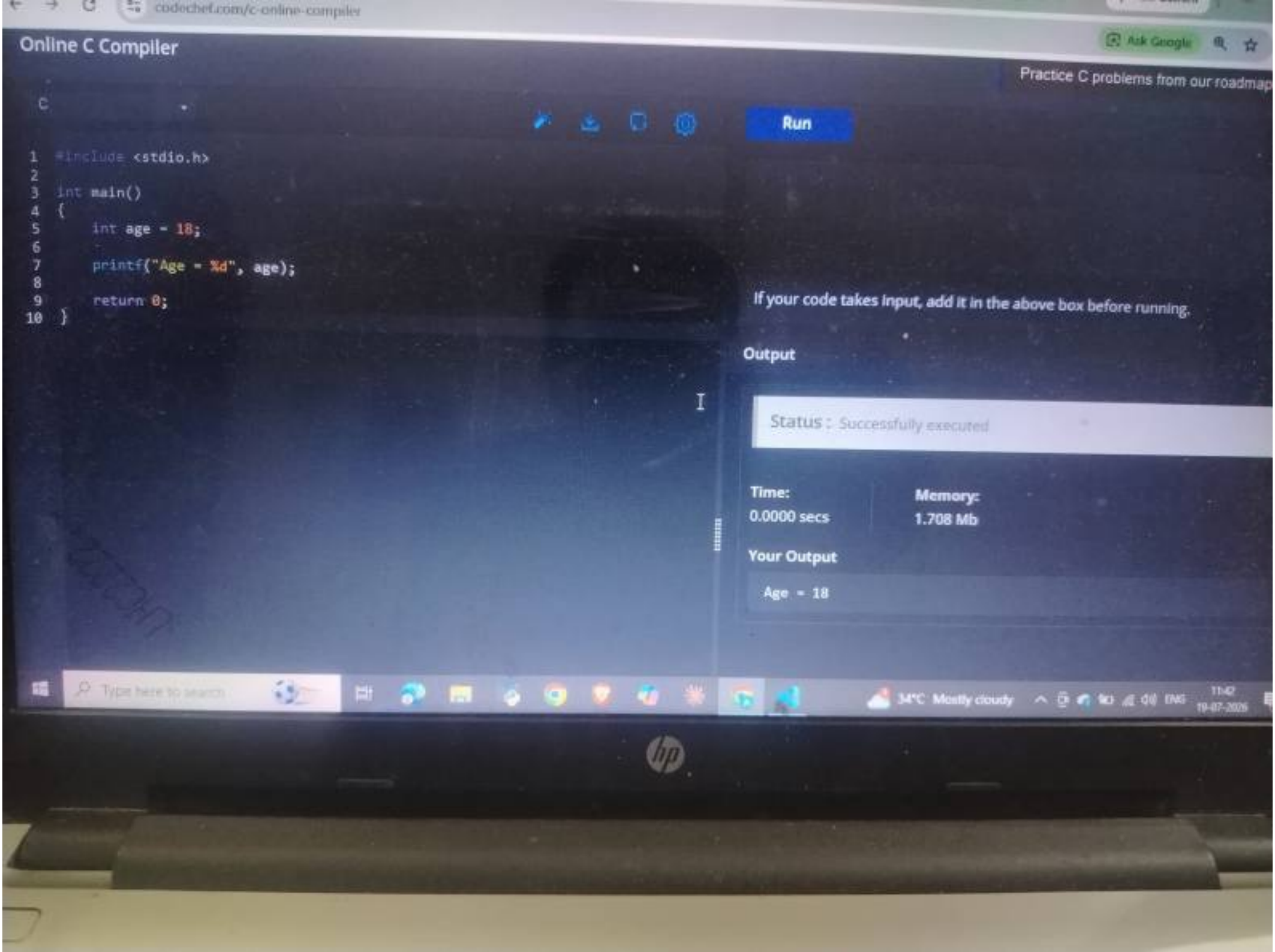
bers

→ Example program in C

```
#include <stdio.h>
int main()
{
    int age = 18;
    printf("Age = %d", age);
    return 0;
}
```

output

Age = 18

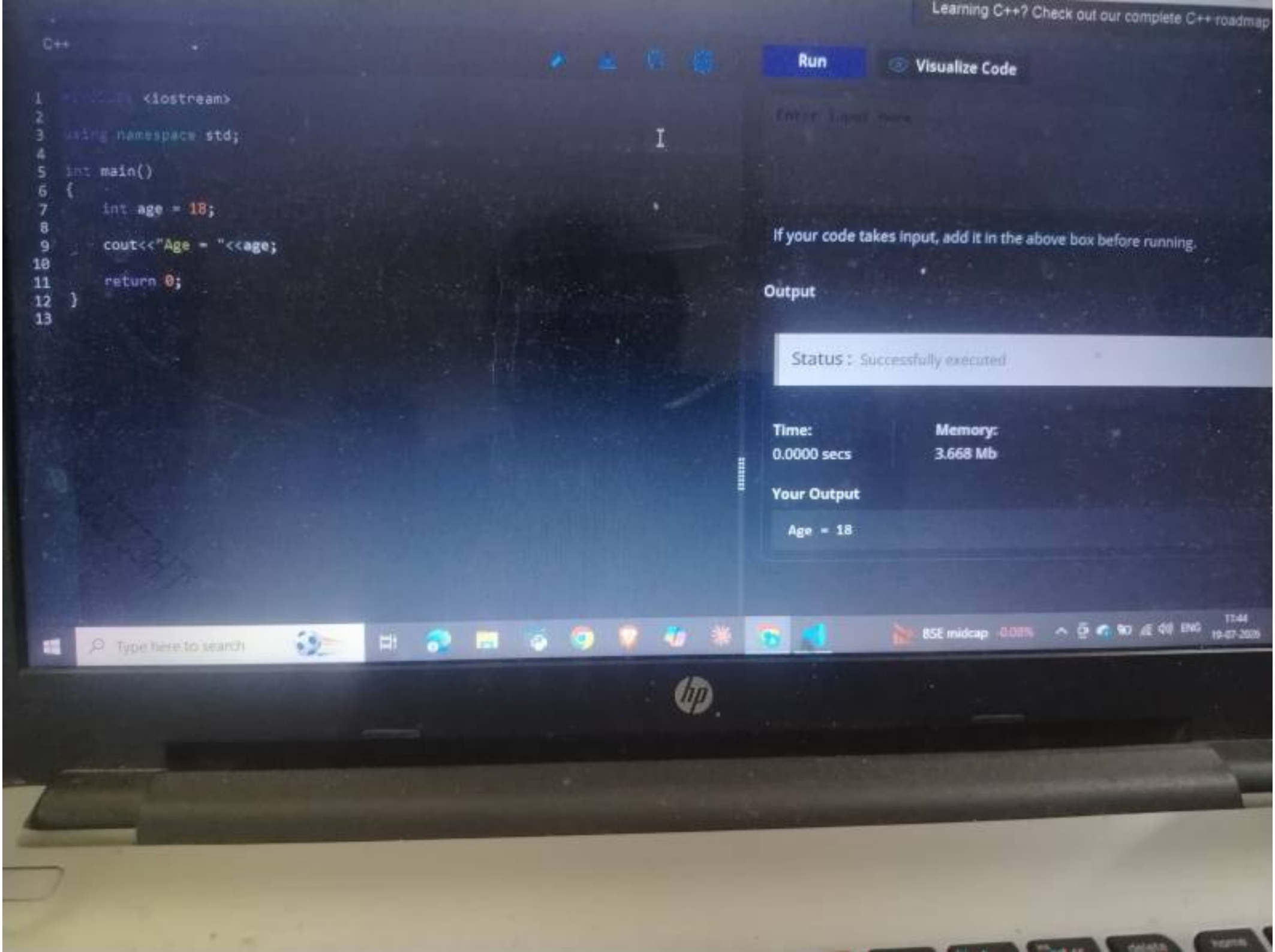

codechef.com/c-online-compiler
Online C Compiler
Practice C problems from our roadmap
C
Run
1 #include <stdio.h>
2
3 int main()
4 {
5 int age = 18;
6
7 printf("Age = %d", age);
8
9 return 0;
10 }
If your code takes input, add it in the above box before running.
Output
Status : Successfully executed
Time:
0.0000 secs
Memory:
1.708 Mb
Your Output
Age = 18
Type here to search
34°C Mostly cloudy
ENG
11:42
19-07-2026

→ Program in C++

```
#include <iostream>

using namespace std;

int main()
{
    cout << " KCC is the best ";

    return 0;

}
```

Output

KCC is the best


Learning C++? Check out our complete C++ roadmap
C++
Run
Visualize Code
#include <iostream>
using namespace std;
int main()
{
int age = 18;
cout<<"Age = "<<age;
return 0;
}
If your code takes input, add it in the above box before running.
Output
Status : Successfully executed
Time:
0.0000 secs
Memory:
3.668 Mb
Your Output
Age = 18
Type here to search
BSE midcap

Conditions and loops in C and C++

Introduction :-

In programming to make decisions or perform the same task repeatedly we use conditions and looping.

Conditions

A condition is a statement that evaluates to either true or false. Based on the result, the program decides which block of code should be executed

Why use conditions

- Make decisions in a program
- Execute code when needed
- Improve program efficiency

PAGE No. 15
DATE :

-> Types of conditional statements

(1) If

executes a block of code only if the specified condition is true

```
if (condition)
 {
    // statement
 }
```

flow

- check the condition
- If the condition is true, execute the statements
- If false skip

(2) if

te ments

only if
true

true,

2) if-else statement

The if else exist when there are two outcomes

If condition is true the if block executes.

Otherwise else block executes

syntax

```
if (condition)
{
    // True block
}
else
{
    //false block
}
```

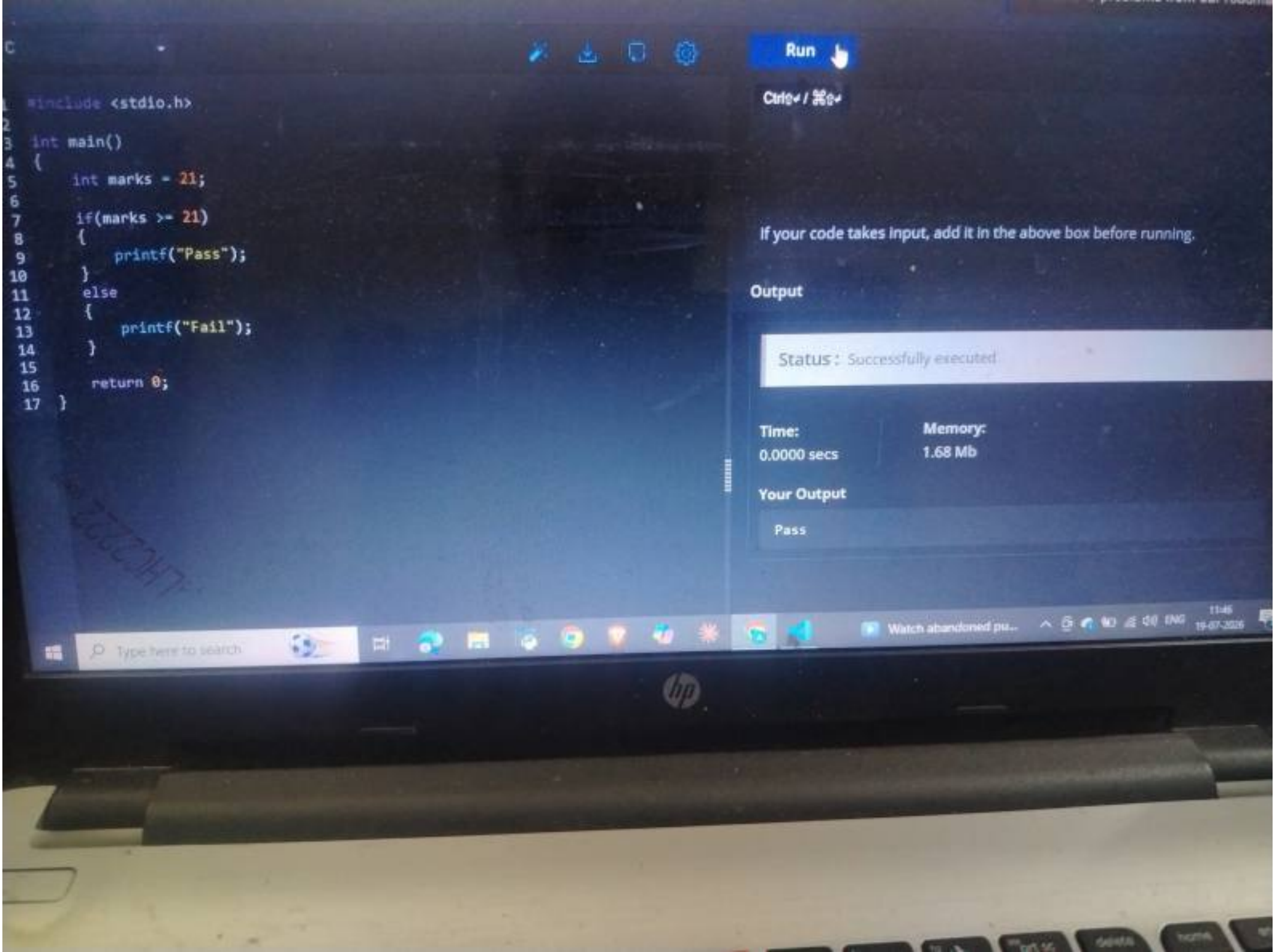
eg

```
#include <stdio.h>
int main()
{
    int marks = 21;
    if ( marks >= 21)
    {
        printf ("Pass");
    }
    else
    {
        printf ("fail");
    }
    return 0;
}
```

output

pass

(3)


C
Run
Ctrl⏎ / ⌘⏎
#include <stdio.h>
int main()
{
int marks = 21;
if(marks >= 21)
{
printf("Pass");
}
else
{
printf("Fail");
}
return 0;
}
If your code takes input, add it in the above box before running.
Output
Status : Successfully executed
Time:
0.0000 secs
Memory:
1.68 Mb
Your Output
Pass
Type here to search
Watch abandoned pu...

(3) else - if ladder

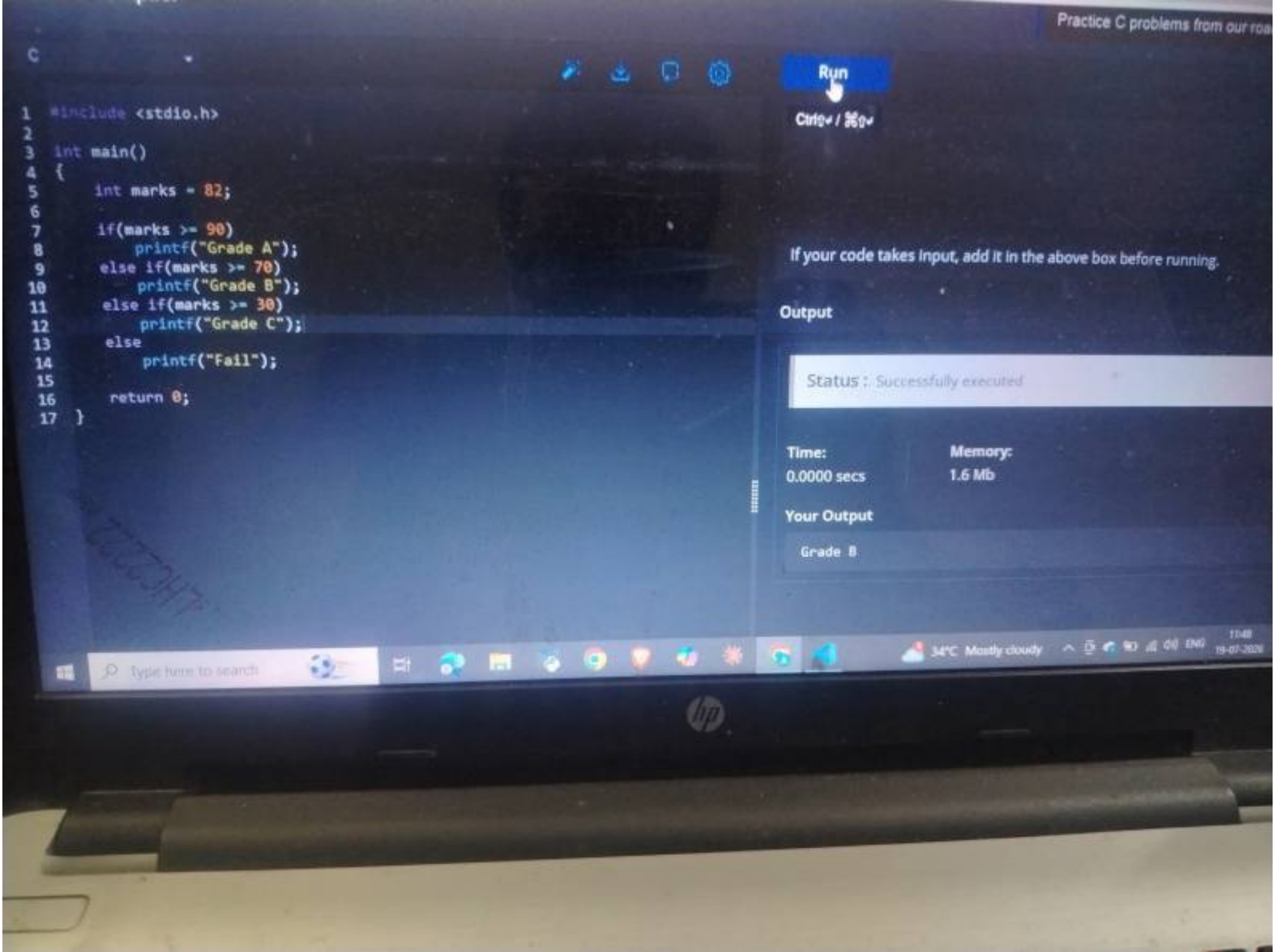
When multiple condition need to be checked

Syntax

```
if (condition1)
{

}
else if (condition2)
{

}

else if (condition 3)
{

}

else
{

}
```

example

```
#include <stdio.h>

int main()
{
    int marks = 82;

    if (marks >= 90)
    {
        printf ("grade A");
    }
    else if (marks >= 70)
    {
        printf ("grade B");
    }

    else if (marks >= 30)
    {
        printf ("grade C");
    }

    else
    {
        printf ("fail");
    }
```


Practice C problems from our roa
C
Run
Ctrl↵ / ⌘↵
1 #include <stdio.h>
2
3 int main()
4 {
5 int marks = 82;
6
7 if(marks >= 90)
8 printf("Grade A");
9 else if(marks >= 70)
10 printf("Grade B");
11 else if(marks >= 30)
12 printf("Grade C");
13 else
14 printf("Fail");
15
16 return 0;
17 }
If your code takes input, add it in the above box before running.
Output
Status : Successfully executed
Time:
0.0000 secs
Memory:
1.6 Mb
Your Output
Grade B
Type here to search
34°C Mostly cloudy
ENG

output :

Grade B

1) Nested if

placing one if statement inside another if

Syntax :-

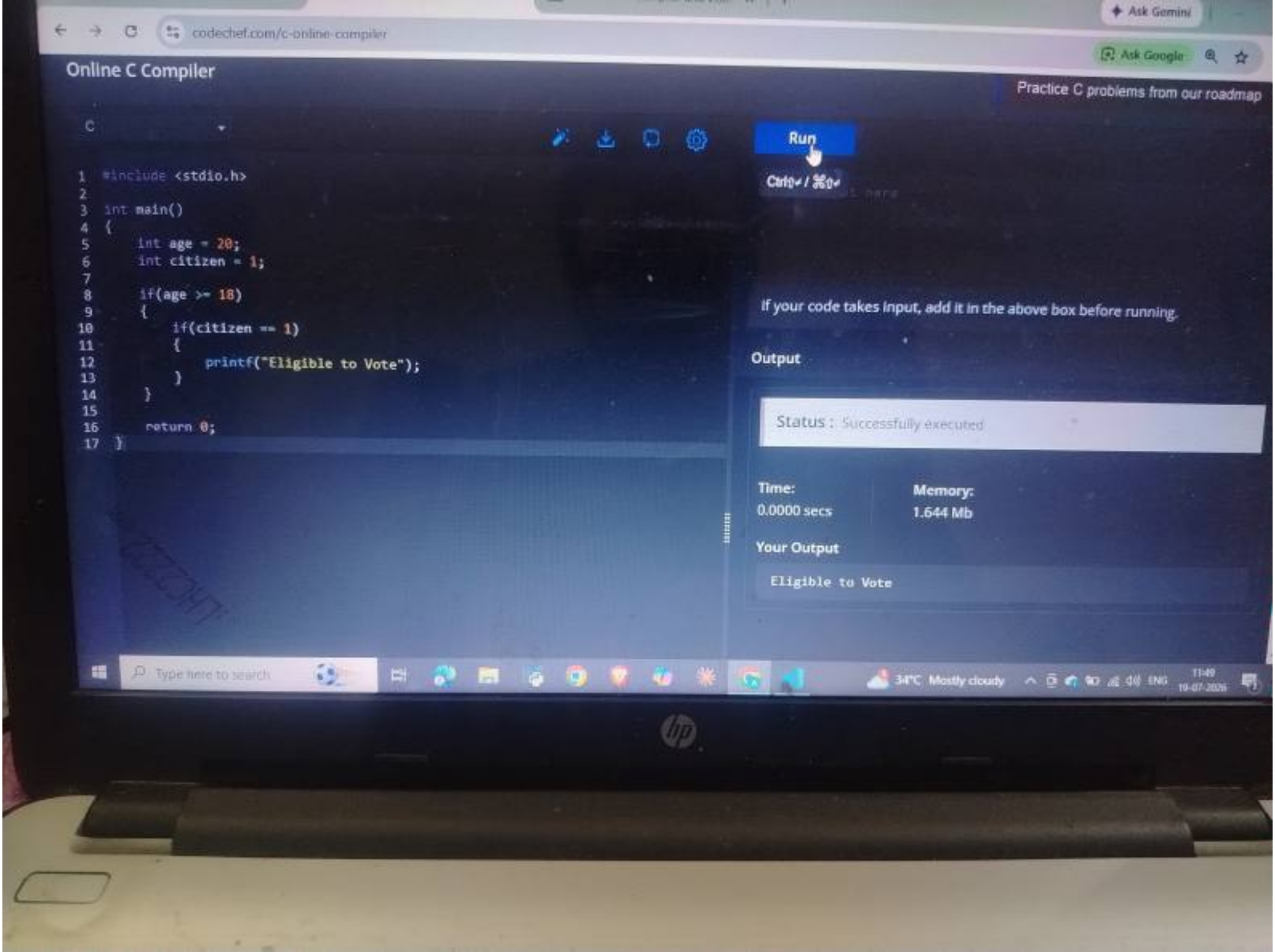
```
if (condition1)
{
    if (condition 2)
    {
    }
}
```

example

```
#include <stdio.h>

int main()
{

    int age = 21;
    int citizen = 1;

    if (age >= 18)
    {
        if (citizen == 1)
        {

            printf("Eligible to vote");

        }

    }

    return 0;

}
```

output

Eligible to vote


codechef.com/c-online-compiler
Ask Gemini
Ask Google
Online C Compiler
Practice C problems from our roadmap
C
Run
Ctrl↵ / ⌘↵
#include <stdio.h>
int main()
{
int age = 20;
int citizen = 1;
if(age >= 18)
{
if(citizen == 1)
{
printf("Eligible to Vote");
}
}
return 0;
}
If your code takes input, add it in the above box before running.
Output
Status : Successfully executed
Time:
0.0000 secs
Memory:
1.644 Mb
Your Output
Eligible to Vote
Type here to search
34°C Mostly cloudy
ENG
11:49
19-07-2026

(5) Switch statement

It is used when one variable has many possible values

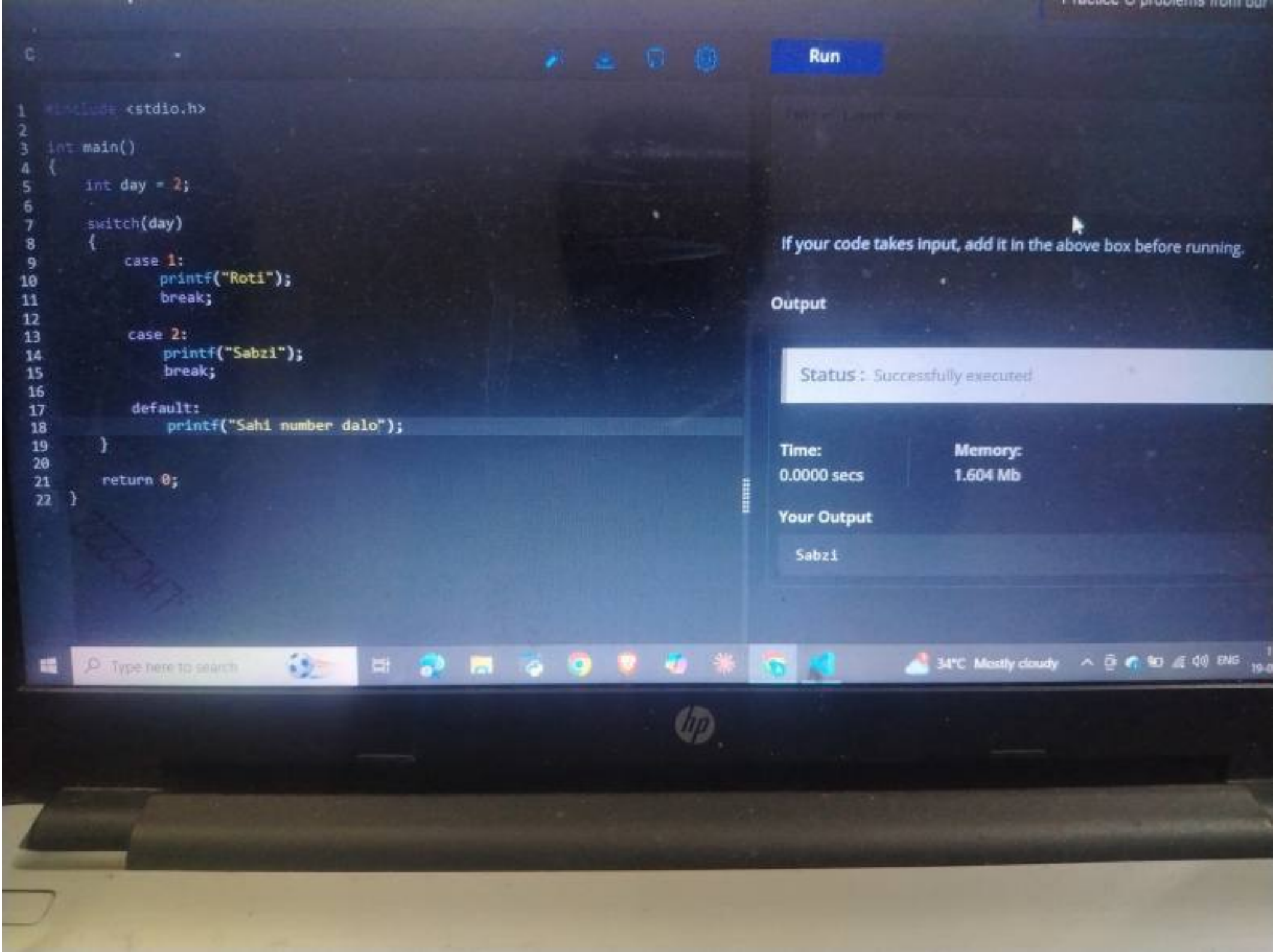
syntax :-

```
Switch (variable)
{
    case value1 :
        statements;
        break;

    case value2 :
        statements;
        break;

    default :
        statements;

}
```

```
#include <stdio.h>

int main()
{
    int item = 2;

    switch(item)
    {
        case 1:
            printf("Roti");
            break;

        case 2:
            printf("Sabzi");
            break;

        default:
            printf("Sahi number dalo");

    }

    return 0;
}
```

Output

Sabzi


C
Run
#include <stdio.h>
int main()
{
int day = 2;
switch(day)
{
case 1:
printf("Roti");
break;
case 2:
printf("Sabzi");
break;
default:
printf("Sahi number dalo");
}
return 0;
}
If your code takes input, add it in the above box before running.
Output
Status : Successfully executed
Time:
0.0000 secs
Memory:
1.604 Mb
Your Output
Sabzi
Type here to search
34°C Mostly cloudy
ENG

- Difference between if-else and switch

if-else	switch
Can check multiple conditions	check only one expression
More flexible	Easy for menu based
Slow for many conditions	faster for multiple fixed values

LOOPS

A loop is a control structure that repeatedly executes a block of code until a specified condition becomes false.

Advantages

- Reduce code repetition
- Saves time
- Easier to maintain
- Improves readability

→ Types of loops

(1) for loop

Where number of iterations is known

Syntax

```
for (Initialization ; condition ;
     increment / decrement)
{
    // statements
}
```

Working

- Initialization executes once.
- Condition is checked

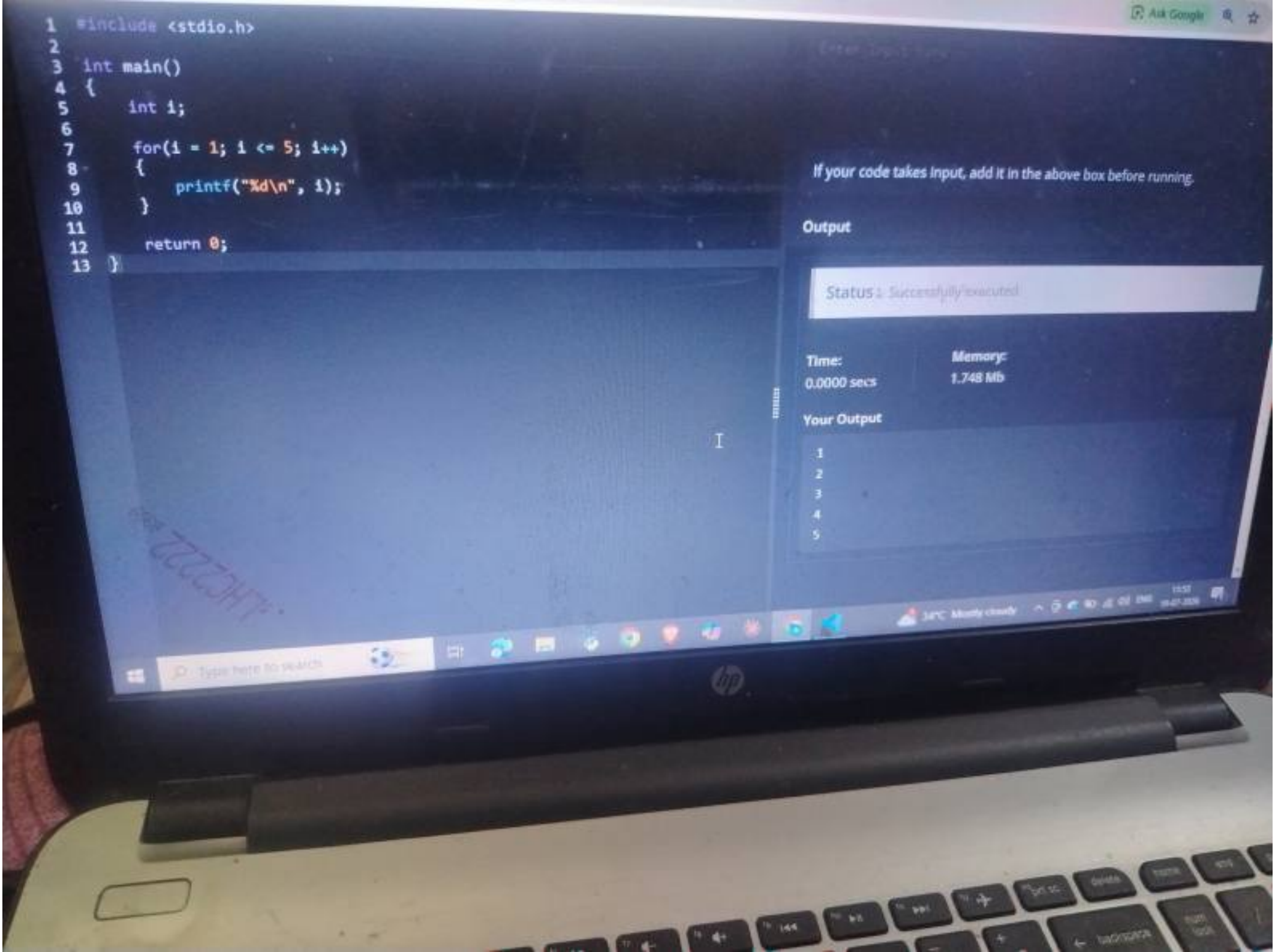
- increment or decrement occurs
- process repeats until the condition becomes false.

Example

```
#include <stdio.h>
int main()
{
    int i;
    for(i=1;i<5;i++)
    {
        printf("%d\n",i);
    }
    return 0;
}
```

output

1
2
3
4
5


#include <stdio.h>
int main()
{
int i;
for(i = 1; i <= 5; i++)
{
printf("%d\n", i);
}
return 0;
}
If your code takes input, add it in the above box before running.
Output
Status : Successfully executed
Time:
0.0000 secs
Memory:
1.748 Mb
Your Output
1
2
3
4
5

(2) While Loop

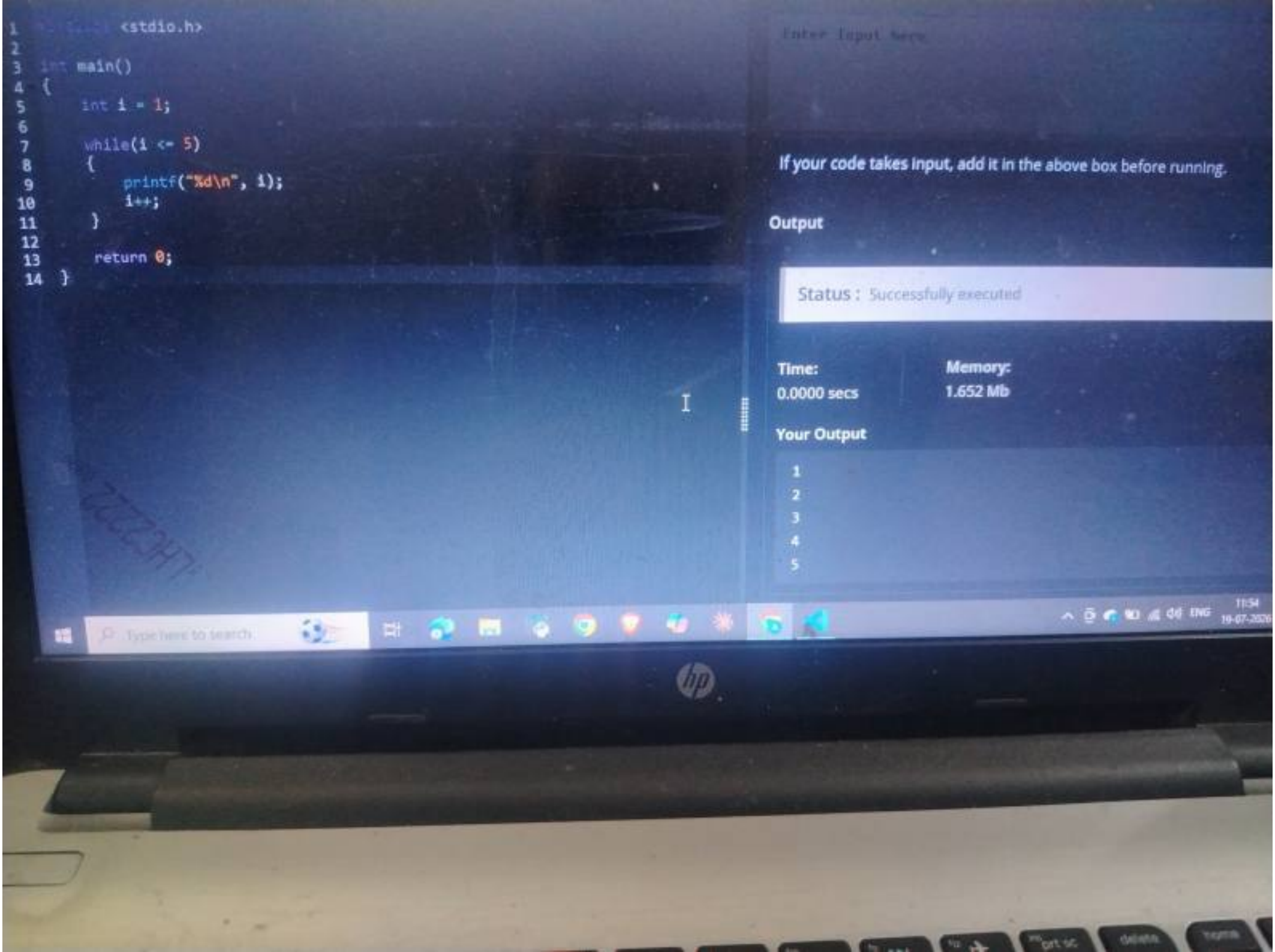
The while loop executes as long as condition is true

Syntax

```
while (condition)
  {
      // statements

  }
```


example

```
#include <stdio.h>

int main()
 {
    int i = 1;

    while (i<= 5)
     {
        printf ("%d\n", i);
        i++;

     }
return 0;
```


#include <stdio.h>
int main()
{
int i = 1;
while(i <= 5)
{
printf("%d\n", i);
i++;
}
return 0;
}
Enter Input Here
If your code takes input, add it in the above box before running.
Output
Status : Successfully executed
Time:
0.0000 secs
Memory:
1.652 Mb
Your Output
1
2
3
4
5

output

1
2
3
4
5

(2) do-while loop

The do-while loop executes the statements at least once, because the condition is checked after the loop body.

Syntax

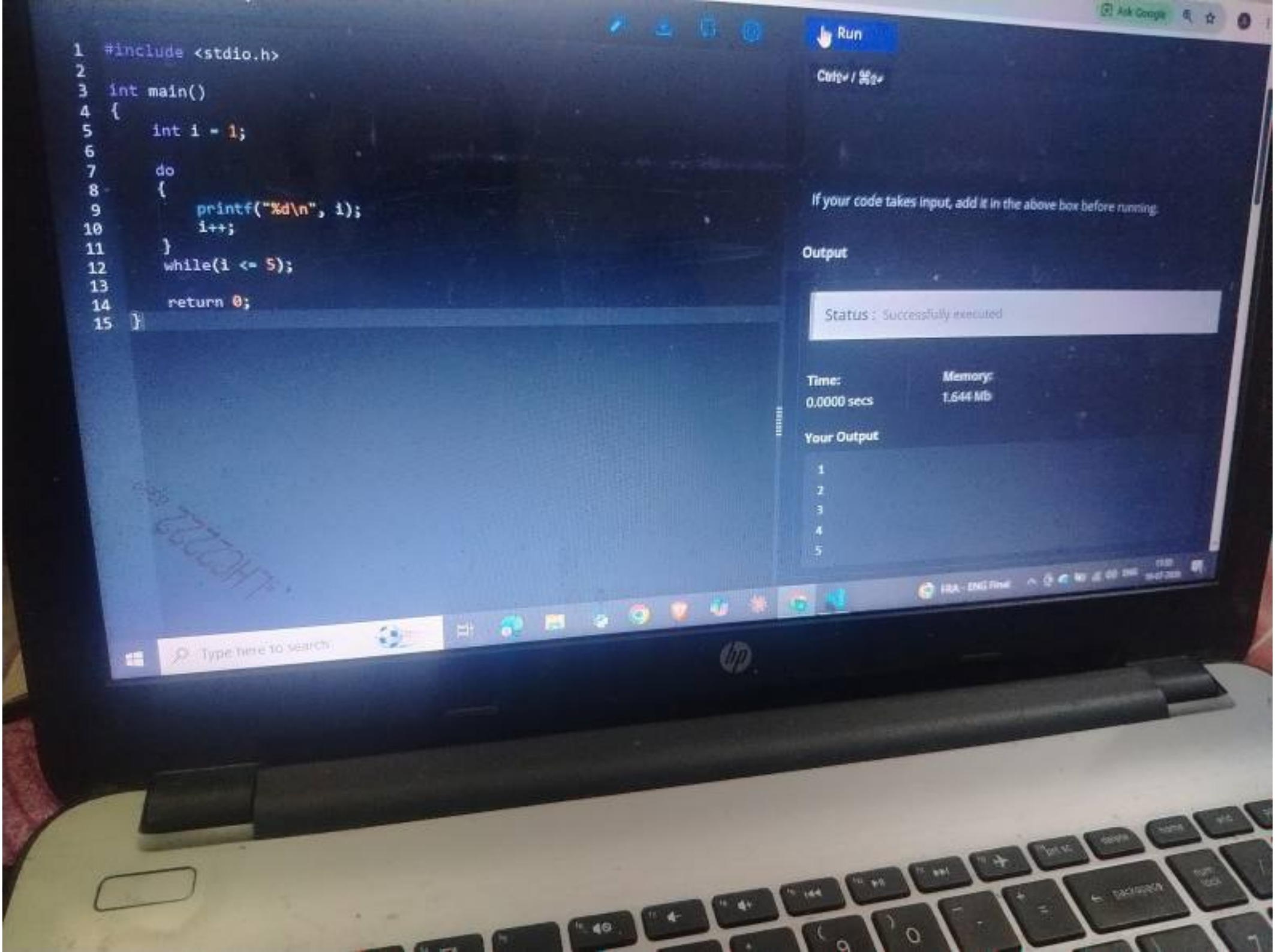
```
do
{
    // statements

}

while (condition);
```

example

```
#include <stdio.h>

int main()
{
    int i = 1;

    do
    {
        printf("%d\n", i);
        i++;

    }

    while (i <= 5);

    return 0;

}
```

output

1
2
3
4
5


#include <stdio.h>
int main()
{
int i = 1;
do
{
printf("%d\n", i);
i++;
}
while(i <= 5);
return 0;
}
Run
If your code takes input, add it in the above box before running.
Output
Status : Successfully executed
Time:
0.0000 secs
Memory:
1.644 Mb
Your Output
1
2
3
4
5
Type here to search

-> Real-life applications of conditions and loops

- Application of conditions

  Atm machine

  Login authentication

  Voting eligibility

- Applications of loops

  Reading files

  processing arrays

  Generating star patterns

-> Advantages of using conditions and loops

Reduces repetitive coding

Saves development time

Improves code readability

Q 1 Square star pattern

Logic

- A square consists of an equal number of rows and columns.
- If the size of the square is 5, then:

outer loop runs 5 times (rows)

Inner loop

Algorithm

(1) Start the program

(2) Declare two variables i and j

(3) Run the outer loop from 1 to 5

(4) Inside it, run the inner loop from 1 to 5

(5) Print *

PAGE No. 31
DATE : / 202

an equal
olumns.

square is 5,

times (rows)

l j

om 1 to 5

er loop

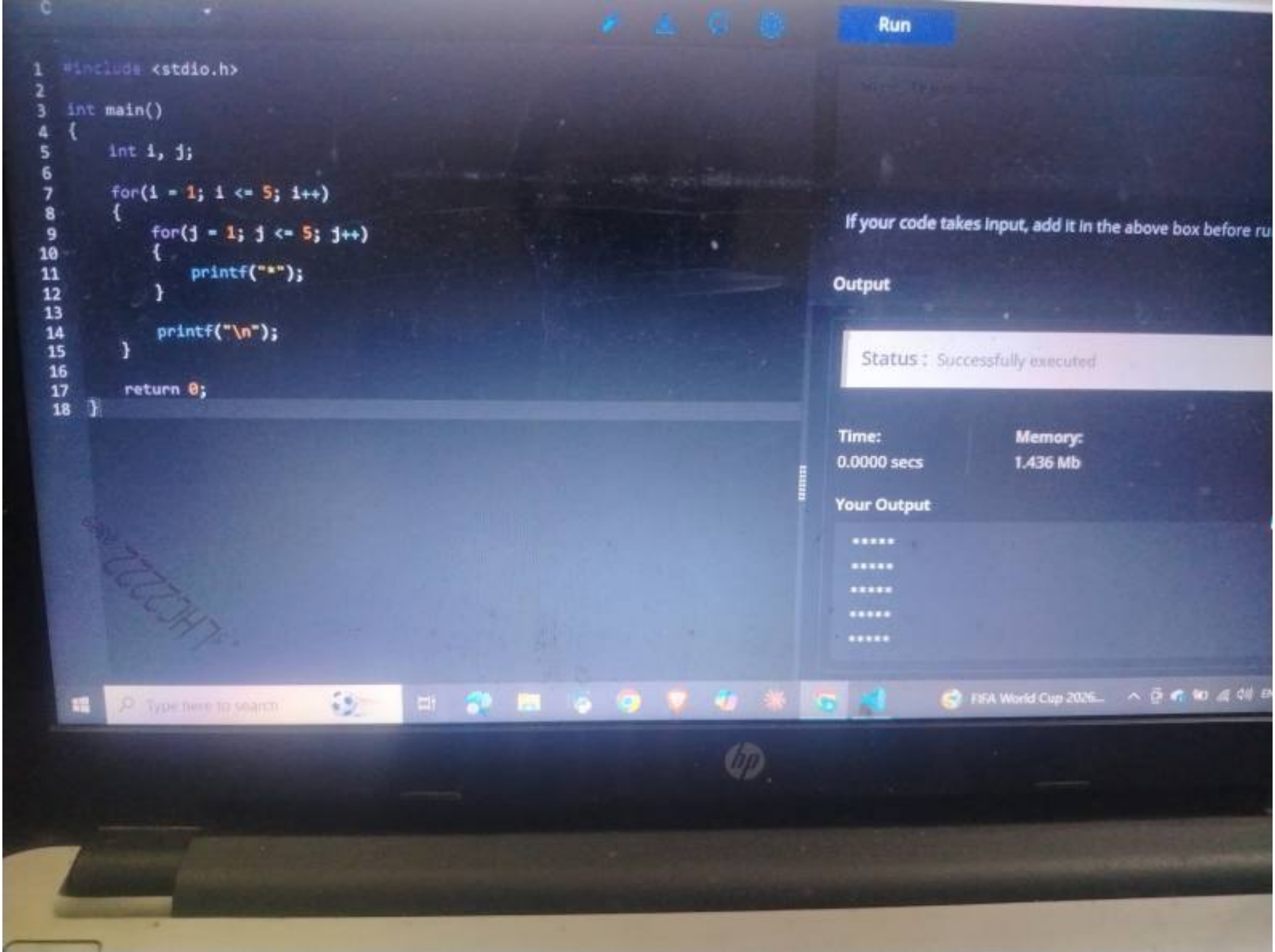
s , print a new

PAGE No. 32
DATE : / 202

```
#include <stdio.h>

int main()
{

int i,j;

for(i=1;i<=5;i++)
{
    for(j=1;j<=5;j++)
    {
        printf("*");
    }

    printf("\n");

}

return 0;
}
```

output

* * * * *
* * * * *
* * * * *
* * * * *
* * * * *


Run
#include <stdio.h>
int main()
{
int i, j;
for(i = 1; i <= 5; i++)
{
for(j = 1; j <= 5; j++)
{
printf("*");
}
printf("\n");
}
return 0;
}
If your code takes input, add it in the above box before ru
Output
Status : Successfully executed
Time:
0.0000 secs
Memory:
1.436 Mb
Your Output
*****
*****
*****
*****
*****
Type here to search
FIFA World Cup 2026...

→ Dry Run

first iteration

i = 1

j = 1 → *
j = 2 → *
j = 3 → *
j = 4 → *
j = 5 → *

output
*****

Similarly i = 2

* * * * *
* * * * *

i = 5

* * * * *
* * * * *
* * * * *
* * * * *
* * * * *

Time complexity $O(n^2)$

Read

Q2

Con

Real life example

- Matrix printing
- Image processing
- Pixel grids
- Game maps

Q2 Hollow Square pattern

pattern

```
* * * * *
*       *
*       *
*       *
* * * * *
```

Concept used

- Nested loops
- if condition

→ Logic

only print stars when

first row, last row, first column, last column

Algorithm

(1) Start

(2) Run outer loop from 1 to 5

(3) Run inner loop from 1 to 5

(4) check

i == 1 or i == 5 or j == 1 or j == 5

if true

Print *

else

print space.

(5) Print next life. line.

```
#include <stdio.h>
int main()
{
    int i,j,k;
    for(i=1;i<=5;i++)
    {
        if(i==1 || i==5 || j==1 || j==5)
            printf("*");
        else
            printf(" ");
    }
    printf("\n");
}
    return 0;
}
```

Output

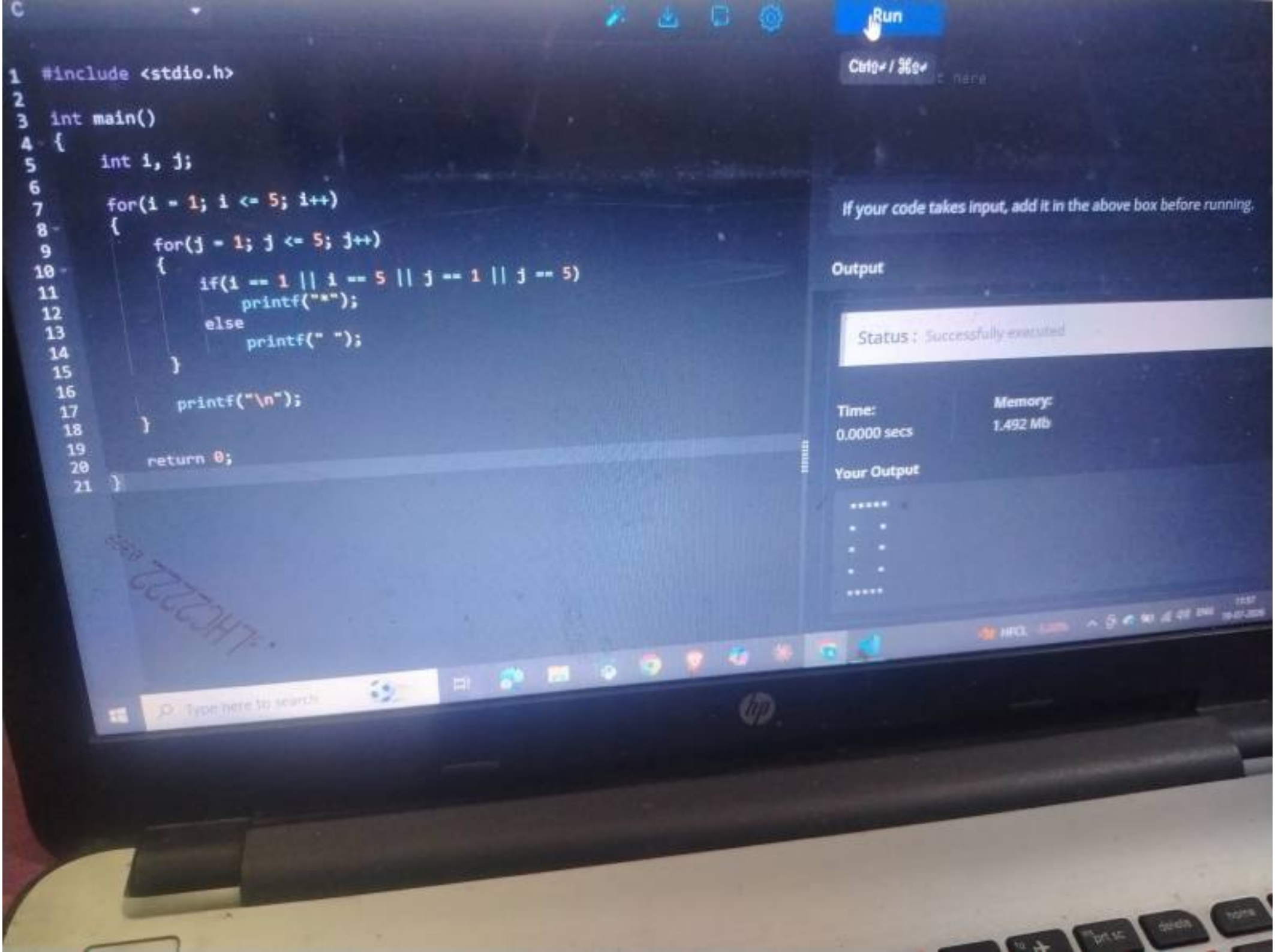
```
* * * * *
*       *
*       *
*       *
* * * * *
```

columns

1 to 5

1 to 5

or j==5


C
Run
Ctrl↵ / ⌘↵
1 #include <stdio.h>
2
3 int main()
4 {
5 int i, j;
6
7 for(i = 1; i <= 5; i++)
8 {
9 for(j = 1; j <= 5; j++)
10 {
11 if(i == 1 || i == 5 || j == 1 || j == 5)
12 printf("*");
13 else
14 printf(" ");
15 }
16
17 printf("\n");
18 }
19
20 return 0;
21 }
If your code takes input, add it in the above box before running.
Output
Status : Successfully executed
Time:
0.0000 secs
Memory:
1.492 Mb
Your Output
*****
* *
* *
* *
*****
Type here to search

Dry run

Row 1

i = 1.

condition true

* * * * *

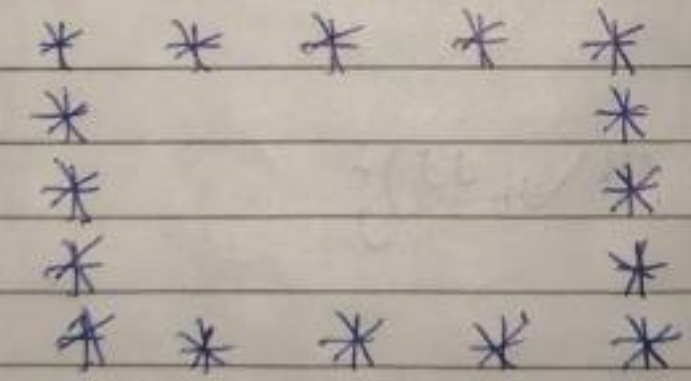
.

.

.

Row 5

i = 5



T.C.

$O(n^2)$

S.C.

$O(1)$

applications

Borders
UI frames

Q3 Right triangle pattern

pattern

```
*
**
***
****
*****
```

Concept used

Nested loops

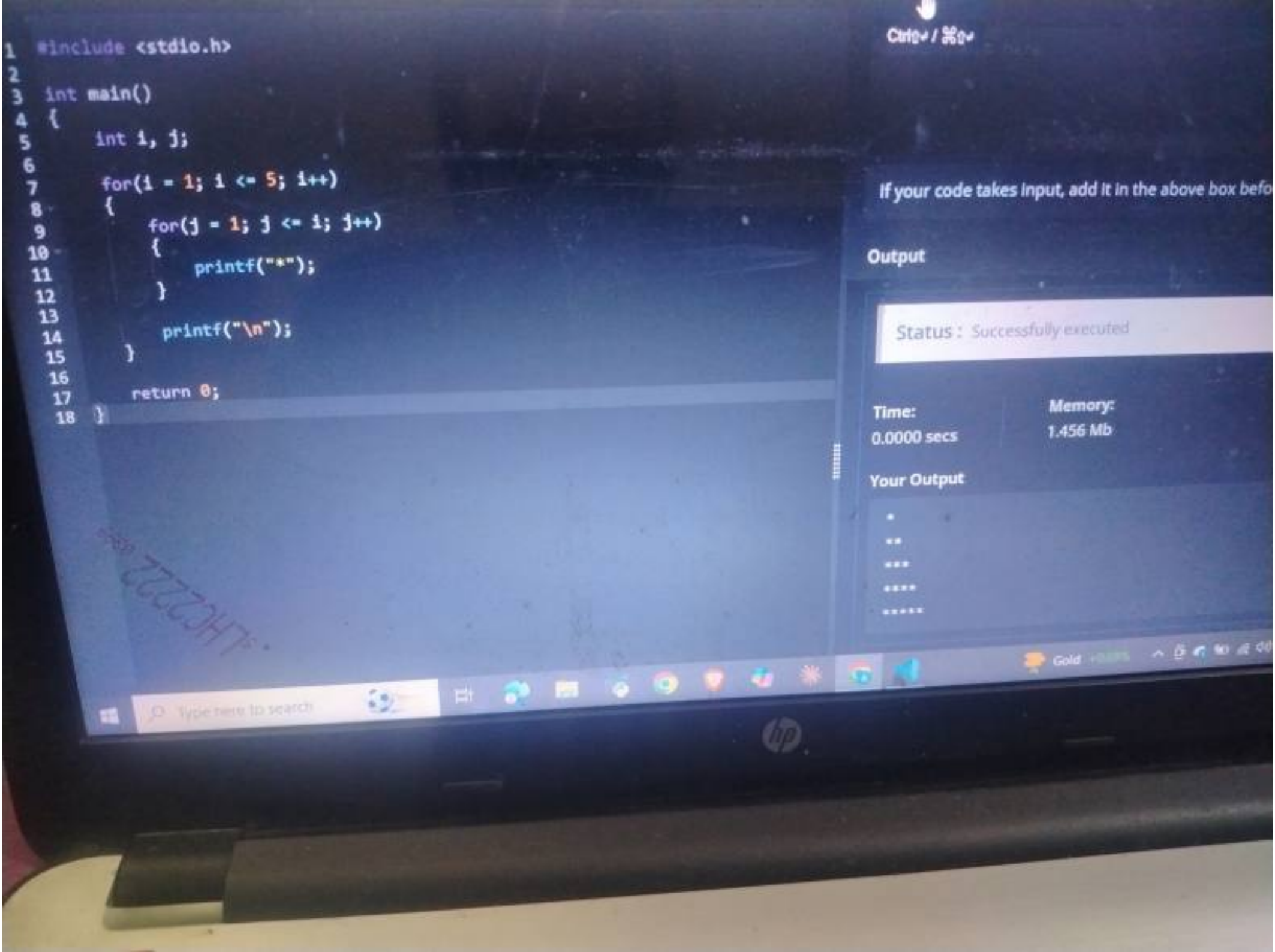
Algorithm

(1) Start
(2) Run outer loop from 1 to 5
(3) Run inner loop from 1 to current row number
(4) Print star
(5) Print new line.

```
#include <stdio.h>

int main()
{
    int i, j;

    for(i=1; i<=5; i++)
    {
        for(j=1; j<=i; j++)
        {
            printf("*");
        }

        printf("\n");

    }

    return 0;

}
```

Output

```
*
**
***
****
*****
```


#include <stdio.h>
int main()
{
int i, j;
for(i = 1; i <= 5; i++)
{
for(j = 1; j <= i; j++)
{
printf("*");
}
printf("\n");
}
return 0;
}
Ctrl↵ / ⌘↵
If your code takes input, add it in the above box befo
Output
Status : Successfully executed
Time:
0.0000 secs
Memory:
1.456 Mb
Your Output
*
**
***
****
*****
Type here to search
Gold

Dry Run

Row 1

i = 1

j = 1

*

1

2

Row 5

i = 5

*****

output

*
**
***
****
*****

T.C.

$O(n^2)$

S.C.

$O(1)$

Applications

- Pattern - based programming
- Nested loop learning
- Computer graphics

Q4: Inverted Triangle pattern

pattern

```
* * * * *
* * * *
* * *
* *
*
```

Algorithm

(1) Start

(2) Run outer loop from 5 down to 1

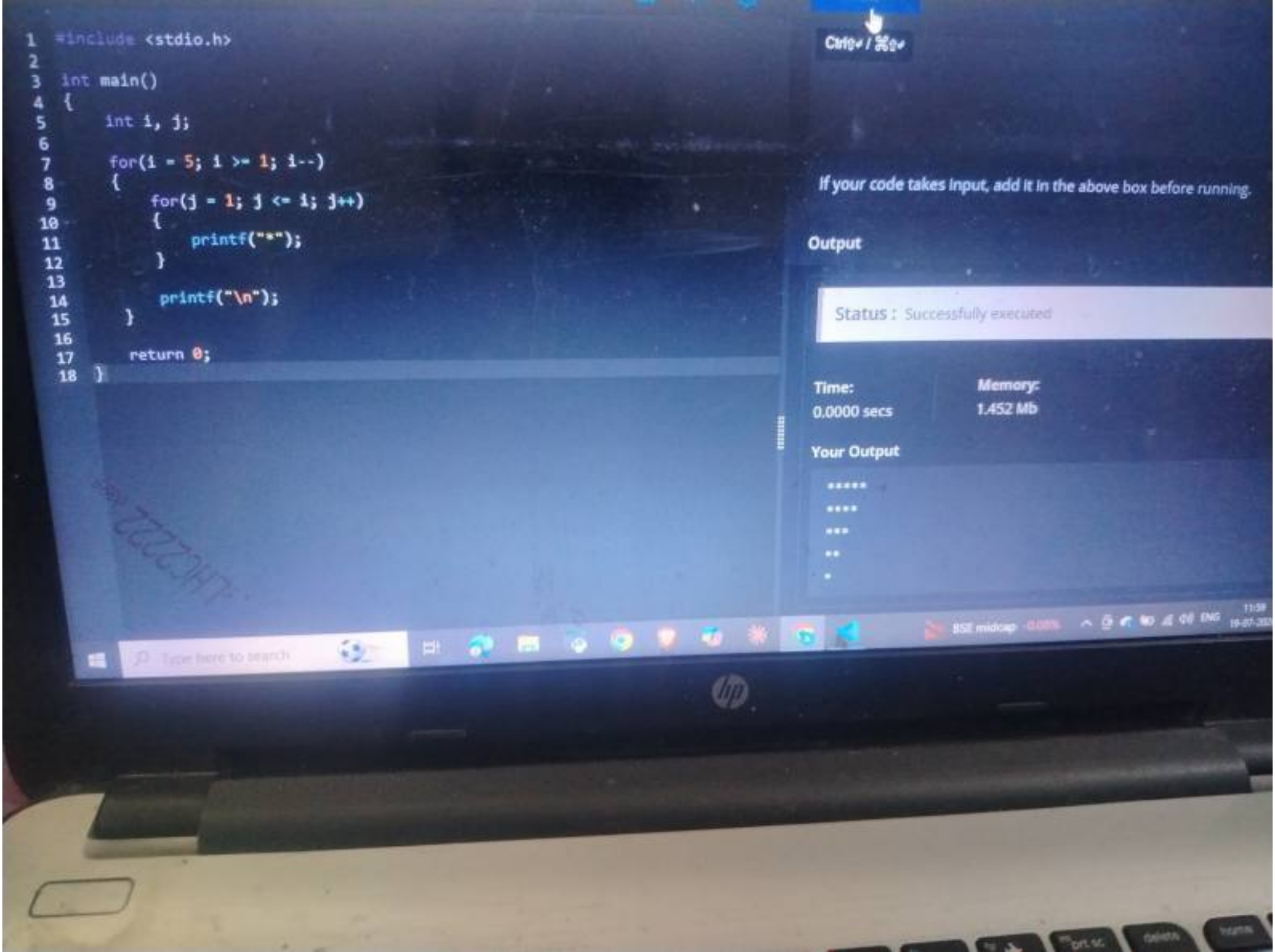
(3) Run inner loop from 1 to current row value

(4) Print stars

(5) Print next line

(6) stop

```
#include <stdio.h>

int main()
{
    int i, j;
    for(i=5; i>=1; i--)
    {
        for(j=1; j<=i; j++)
        {
            printf("*");
        }
        printf("\n");
    }
    return 0;
}
```



```c
#include <stdio.h>

int main()
{
    int i, j;

    for(i = 5; i >= 1; i--)
    {
        for(j = 1; j <= i; j++)
        {
            printf("*");
        }

        printf("\n");
    }

    return 0;
}
```

Ctrl↵ / ⌘↵

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0000 secs

Memory: 1.452 Mb

Your Output

```
*****
****
***
**
*
```

Output

```
* * * * *
* * * *
* * *
* *
*
```

Dry run

Row 1

i = 5

j = 1 to 5

* * * * *

1

1

1

Row 5

i = 1

*

(1)
(2)
(3)
(4)
(5)
(6)
(7)

- T.C. $O(n^2)$

S.C. $O(1)$

Q5 Pyramid pattern

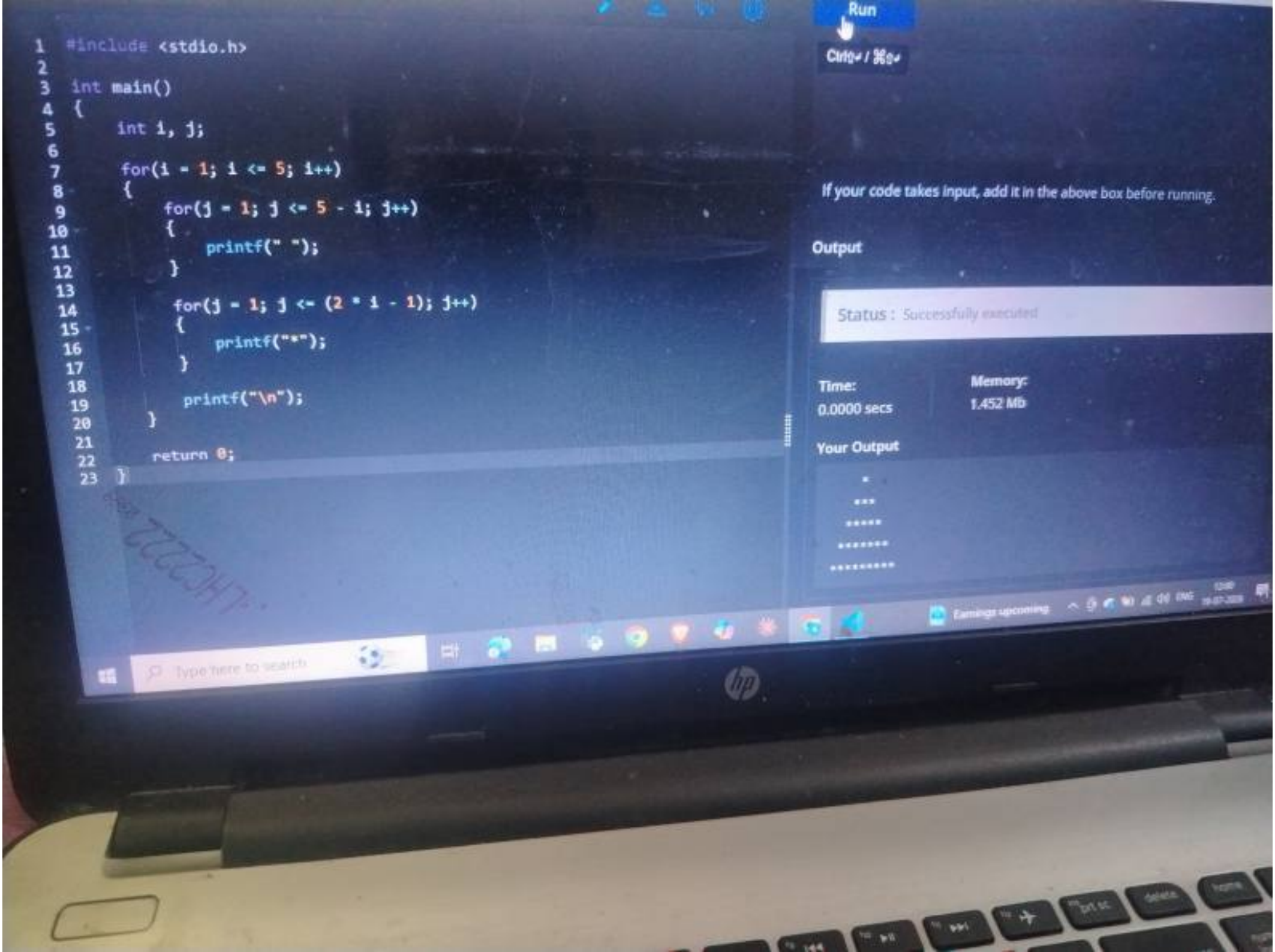
Concept used

- Nested for loops
- Printing spaces
- printing stars

Algorithm

(1) Start the program
(2) Declare variables i and j
(3) Run the outer loop from 1 to 5
(4) Print spaces (5-i)
(5) print stars (2*i-1)
(6) Move to the next line
(7) Stop

```
#include <stdio.h>

int main()
{
    int i, j, k;

    for (i=1; i<=5; i++)
    {
        for(j=1; j<=5-i; j++)
        {
            printf("  ");
        }

        for (k=1; k<=(2*i-1); k++)
        {
            printf("*");
        }

        printf("\n");

    }

    return 0;

}
```



```c
#include <stdio.h>

int main()
{
    int i, j;

    for(i = 1; i <= 5; i++)
    {
        for(j = 1; j <= 5 - i; j++)
        {
            printf(" ");
        }

        for(j = 1; j <= (2 * i - 1); j++)
        {
            printf("*");
        }

        printf("\n");
    }

    return 0;
}
```

Run

Ctrl↵ / ⌘↵

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0000 secs

Memory: 1.452 Mb

Your Output

```
*
***
*****
*******
*********
```

Dry run

Row 1

r = 1

Spaces = 4
Stars = 1

2
1
:
:

Row 5

i = 5

Spaces = 0
Stars = 9

* * * * *

output

T.C -

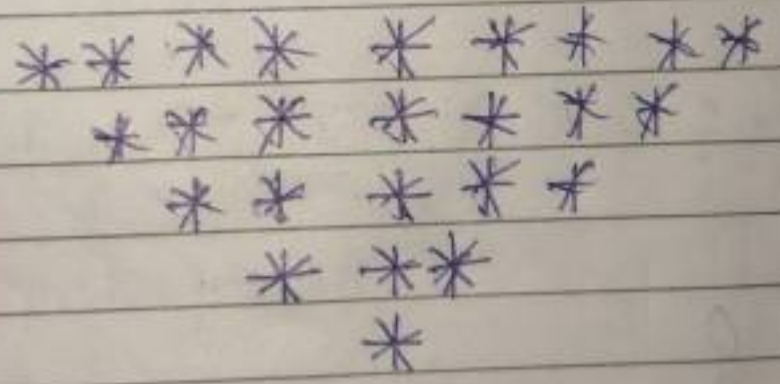
$O(n^2)$

S.C -

$O(1)$

Q6 Inverted pyramid Pattern

pattern



Concept used

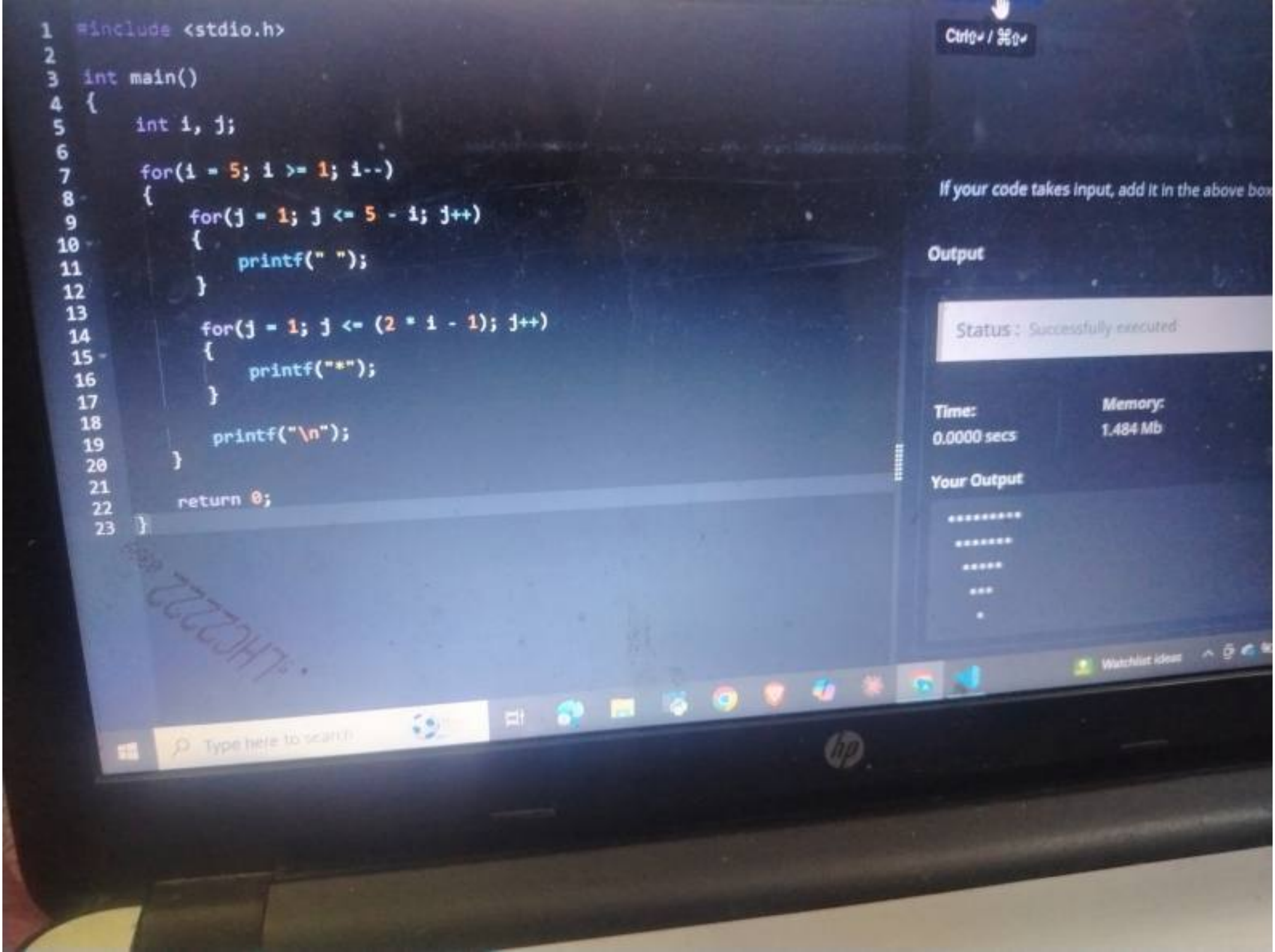
- Nested loops
- printing spaces
- printing stars

PAGE NO: 48
DATE:

Algorithm

(1) Start
(2) Outer loop runs from 5 down to 1
(3) print required spaces
(4) print required stars
(5) Move to next line.
(6) Stop

```
#include <stdio.h>

int main()
{
    int i,j,j;
    for (i=5; i>=1; i--)
    {
      for(j=1; j<=5-i; j++)
        {
            printf(" ");
        }

      for(j=1; j<=(2*i-1); j++)
        {
            printf ("*");
        }
      printf("\n");
    }
    return 0;
}
```



```c
#include <stdio.h>

int main()
{
    int i, j;

    for(i = 5; i >= 1; i--)
    {
        for(j = 1; j <= 5 - i; j++)
        {
            printf(" ");
        }

        for(j = 1; j <= (2 * i - 1); j++)
        {
            printf("*");
        }

        printf("\n");
    }

    return 0;
}
```

Ctrl↵ / ⌘↵

If your code takes input, add it in the above box

Output

Status : Successfully executed

Time:
0.0000 secs

Memory:
1.484 Mb

Your Output

```
*********
*******
*****
***
*
```

Type here to search

Dry run

Row 1 i=5, space=0, stars=9

* * * * * * * * *

Row 5 i=1, spaces=4, stars=1

output

* * * * * * * * *
 * * * * * * *
  * * * * *
   * * *
    *

T.C. $O(n^2)$

S.C. $O(1)$

Q7 print half diamond pattern

Algorithm

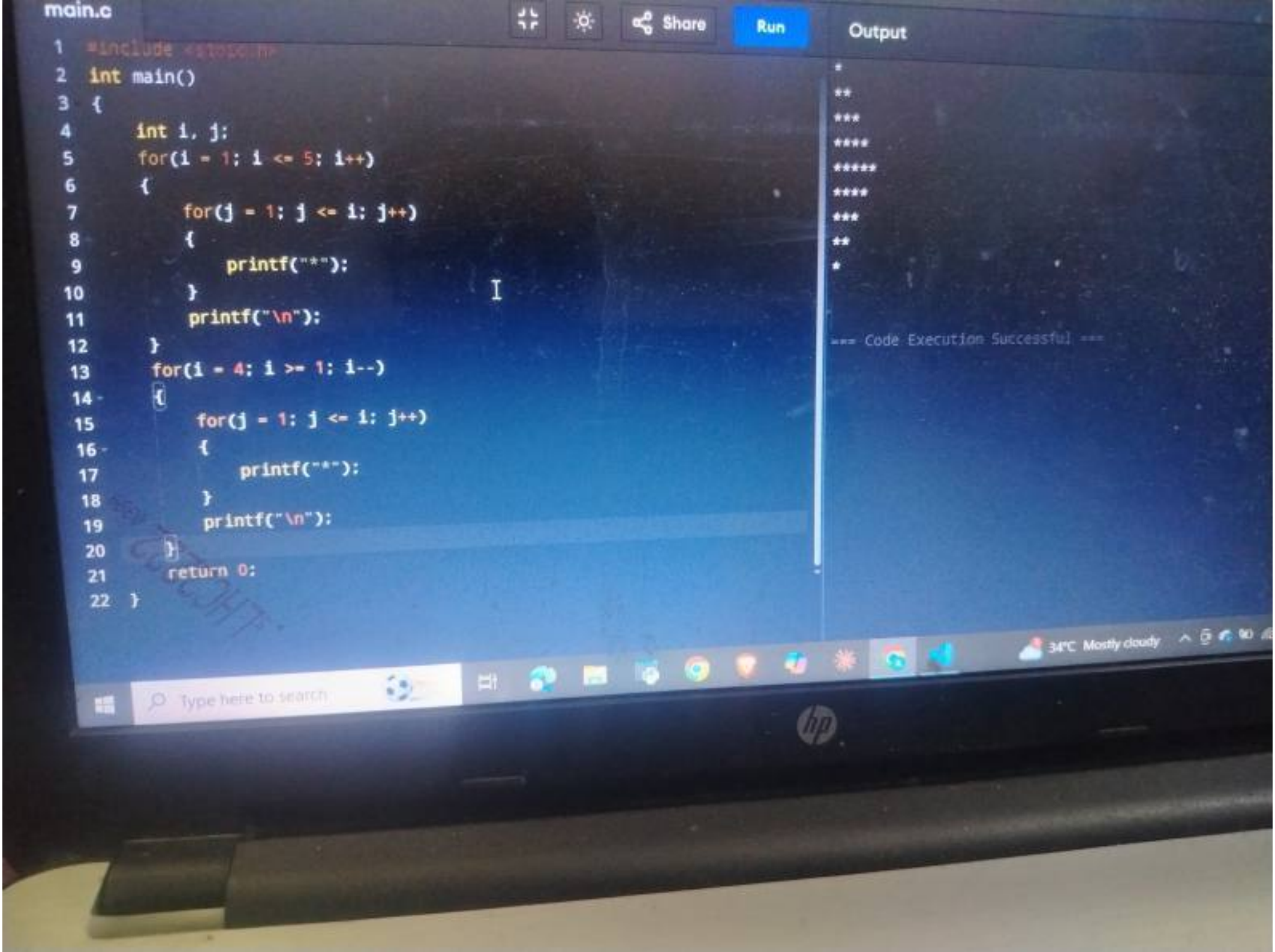
(1) Start
(2) print increasing triangle
(3) print decreasing triangle
(4) stop

```
#include<stdio.h>
int main()
{
  int i,j,j;
  for(i=1;i<=5;i++)
  {
    for(j=1;j<=i;j++)
    {
      Printf("*");
    }
    printf("\n");
  }
  for(i=4;i>=1;i--)
  {
    for(j=1;j<=i;j++)
    {
      printf("*");
    }
    printf("\n");
  }
  return 0;
}
```

Output

upper half

```
i=1
*
:
i=5
* * * * *
```


main.c
Share
Run
Output
#include <stdio.h>
int main()
{
int i, j;
for(i = 1; i <= 5; i++)
{
for(j = 1; j <= i; j++)
{
printf("*");
}
printf("\n");
}
for(i = 4; i >= 1; i--)
{
for(j = 1; j <= i; j++)
{
printf("*");
}
printf("\n");
}
return 0;
}
*
**
***
****
*****
****
***
**
*
=== Code Execution Successful ===
Type here to search
34°C Mostly cloudy

Lower half

i=4

** **

:

:

:

i=1

*

output

```
*
**
***
****
*****
****
***
**
*
```

T.C. $O(n^2)$

S.C. $O(1)$

Q8 Diamond pattern

```
#include <stdio.h>

int main()
{
    int i,j,j;
    for(i=1;i<=5;i++)
    {
        for(j=1;j<=5-i;j++)
        {
            printf("  ");
        }
```

```
for(j=1; j<=(2*i-1); j++)
{
    printf("*");
}

printf("\n");
}
for(i=4; i>=1; i--)
{
for(j=1; j<=5-i; j++)
{
    printf("*");
}

printf("\n");
}
return 0;
}
```

Dry Run

Upperhalf

i = 1
*

i = 2
***

i = 3
*****

i = 4
****

i = 5
****

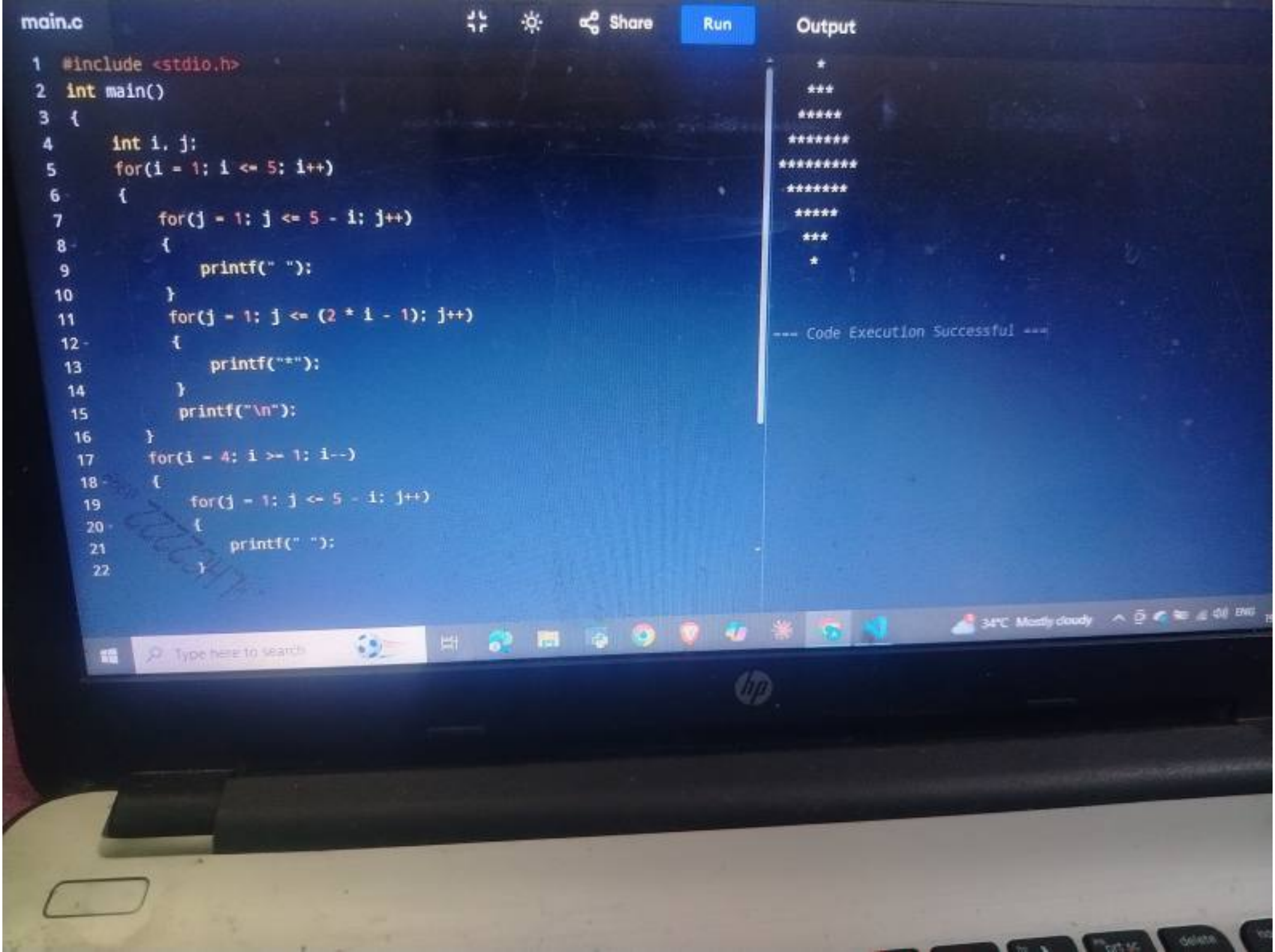
Lowerhalf

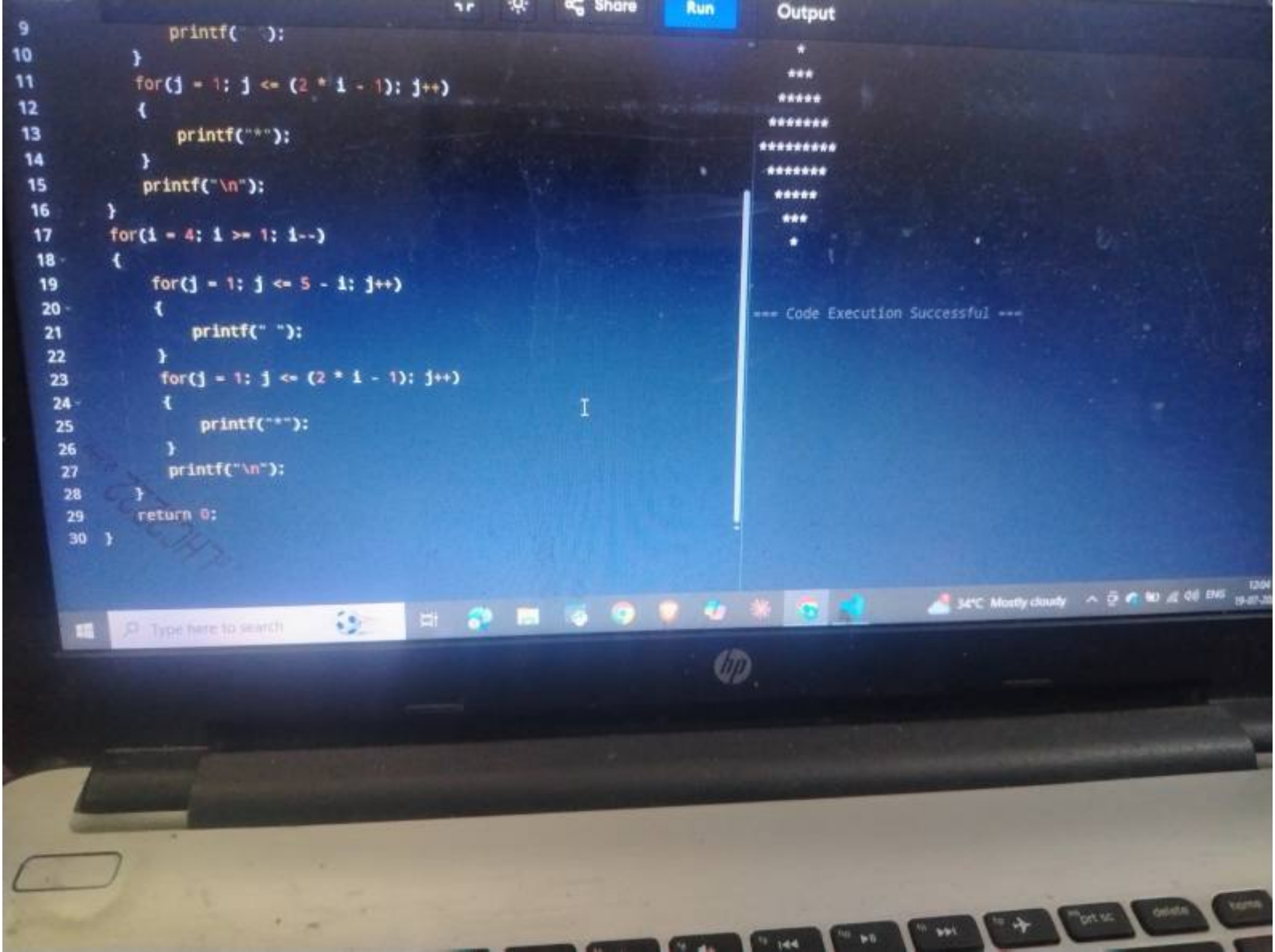
i = 4
****

i = 3
*****

i = 2
***

i = 1
*


main.c
Share
Run
Output
#include <stdio.h>
int main()
{
int i, j;
for(i = 1; i <= 5; i++)
{
for(j = 1; j <= 5 - i; j++)
{
printf(" ");
}
for(j = 1; j <= (2 * i - 1); j++)
{
printf("*");
}
printf("\n");
}
for(i = 4; i >= 1; i--)
{
for(j = 1; j <= 5 - i; j++)
{
printf(" ");
}
*
***
*****
****
****
****
*****
***
*
=== Code Execution Successful ===
Type here to search
34°C Mostly cloudy


Share
Run
Output
printf( );
}
for(j = 1; j <= (2 * i - 1); j++)
{
printf("*");
}
printf("\n");
}
for(i = 4; i >= 1; i--)
{
for(j = 1; j <= 5 - i; j++)
{
printf(" ");
}
for(j = 1; j <= (2 * i - 1); j++)
{
printf("*");
}
printf("\n");
}
return 0;
}
*
***
*****
****
****
****
*****
***
*
=== Code Execution Successful ===
Type here to search
34°C Mostly cloudy